



CISC vs. RISC



CISC History

- Until the early 1980s, all CPUs, whether single-chip or whole-board, followed the CISC (Complex instruction set computer) design philosophy.
- CISC refers to CPUs with hundreds of instructions designed for every possible situation.
- To design CPUs with so many instructions consumed not only hundreds of thousands of transistors, but also made the design very complicated, time-consuming, and expensive.



CISC History

- Memory in those days was expensive
 - bigger program->more storage->more money
- Hence needed to *reduce* the number of instructions per program
- Number of instructions are reduced by having *multiple operations* within a single instruction
- Multiple operations lead to many different kinds of instructions that access memory
 - In turn making instruction length variable and fetch-decode-execute time unpredictable – making it more complex
 - Thus hardware handles the complexity
- Example: x86 ISA



RISC History

- In the early 1980s, a new CPU design philosophy called RISC (reduced instruction set computer) was developed.
- The proponents of RISC argued that no one was using all the instructions by simplifying and reducing them from hundreds to around 40 or so and use all the transistors that are saved to enhance the scientists at IBM as early as the 1970s.
- The first working single-chip RISC microprocessor was implemented by a group of researchers at the University of California at Berkeley in 1980.



- Reduced Instruction Set Architecture (RISC) emerged around early 80s
 - Designers re-evaluating the current ISAs of the era
 - Found that ISAs had extensive instructions that were complex
 - Complex Instruction Set Architecture (CISC)
 - Need only 20% of the instructions that were used most of the time



Introduction - RISC and CISC

- Boundaries have blurred.
- Modern CPUs Utilize features of both.
- The Manufacturing and Economics aspect.



Debate becoming moot

- Converging implementations , example
 - Typical RISC features :
 - Fewer Instructions
 - Fixed instruction length
 - Fixed execution time
 - Lower Cost
 - No longer restricted to RISC.



Historical Context

- Design approaches developed around available technological resources.
 - Memory - expensive
 - Compilers - lousy
 - VLSI - primitive



No Big Difference Now!

- Common Goal of High Performance will bring them together
 - Incorporating each other's features
 - Incorporating similar functional units.
 - Branch Prediction
 - Out of Order Execution etc



An exception

- Embedded Processors
 - CISC is unsuitable
 - MIPS/watt ratio
 - Power consumption
 - Heat dissipation
 - Simple Hardware = integrated peripherals



CISC to RISC (1)

- What Intel, the most famous CISC advocates, and HP do in IA-64:
 - Migrate to a Common Instruction Set.
 - Creating Small Instructions
 - More concise Instruction Set.
 - Shorter Pipeline
 - Lower Clock Cycle



CISC to RISC (2)

- What Intel, the most famous CISC advocates, and HP do in IA-64:
 - Abandon the Out-of-order Execution In Hardware
 - Depend on Compiler to Handle Instruction Execution Order. Shifting the Complexity to Software.



CISC to RISC (3)

- CISC Datapaths Support Other RISC-like Features (such as register-to-register addressing and an expanded register count).



RISC to CISC (1)

- Additional registers
- On-chip caches (which are clocked as fast as the processor)
- Additional functional units for superscalar execution



RISC to CISC (2)

- Additional "non-RISC" (but fast) instructions
- On-chip support for floating-point operations
- Increased pipeline depth



CISC and RISC

- Incorporating Same Features
 - Complex Multi-level Cache
 - Branch Prediction
 - Out-of-order Execution



CISC vs. RISC (In General !!)

- The major difference between CISC and RISC is CISC computers use many instructions whereas RISC computers use a small set of instructions.



CISC vs RISC

- Hard to Distinguish Now. Boundary is getting vague.
- Academia don't Care
- Industry doesn't Care (Except for Advertisements)



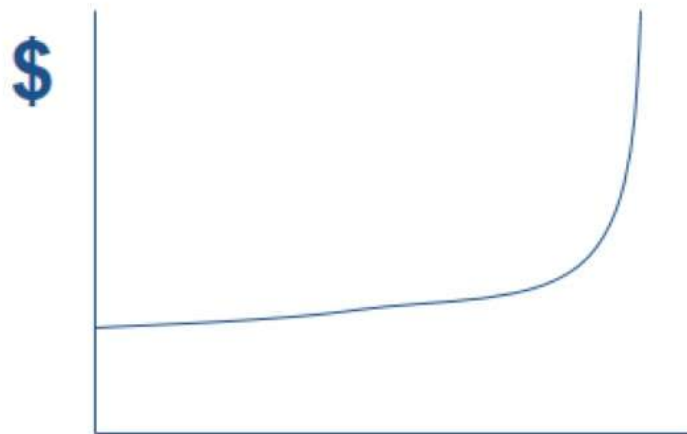
RISC vs CISC

- Which one is better for general-purpose microprocessor design?
- It does not matter because
 - The main factor driving general-purpose microprocessor design has been the peculiar economics of semiconductor manufacturing



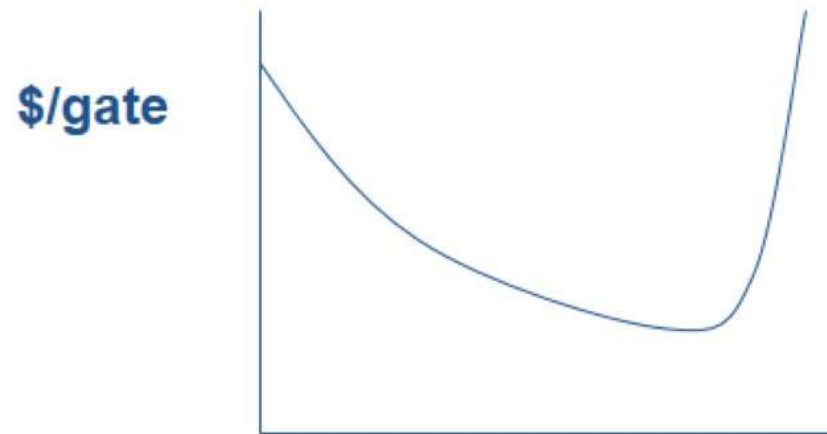
Economics of IC Manufacturing

Cost per chip



Transistor count

Cost per transistor



Transistor count



The graph tells us...

- These curves strongly favor designs near the knee of the curve
- All microprocessors in a certain time have roughly the same number of transistors
- Key design tradeoff: what to do with a given number of transistors?



RISC vs CISC: 500k transistors

- For a few years in the late 80's, designers had a choice:
 - CISC CPU and no on-chip cache
 - RISC CPU and on-chip cache
- On-chip cache was probably a slightly better choice, giving RISC several years of modest advantage
- It is not RISC who gave better performance at this certain period; it was about the on-chip cache!



RISC vs CISC: 2M transistors

- Now possible to have both CISC and on-chip cache
- CISC can challenge RISC and it even has more advantage
- RISC chips become more CISC-like



Even More Transistors

- Then more transistors became available than single CISC CPU and reasonable cache could use... What now?
 - Multi-processor chips?
 - Superscalar?
 - VLIW?



Convergence: 5M transistors

- Superscalar won. But
 - It is really hard to pipeline and schedule superscalar computations when instruction cycles, word-lengths differ, and when there are 100s of different instructions
 - Compilers used only a small subset of instructions
- This pushed CISC designs to be more RISC-like



Even more: 50M transistors

- The economy of IC manufacturing have been making RISC and CISC go together
- Maybe one day these two become historic terms and ?ISC will prevail



Case Study

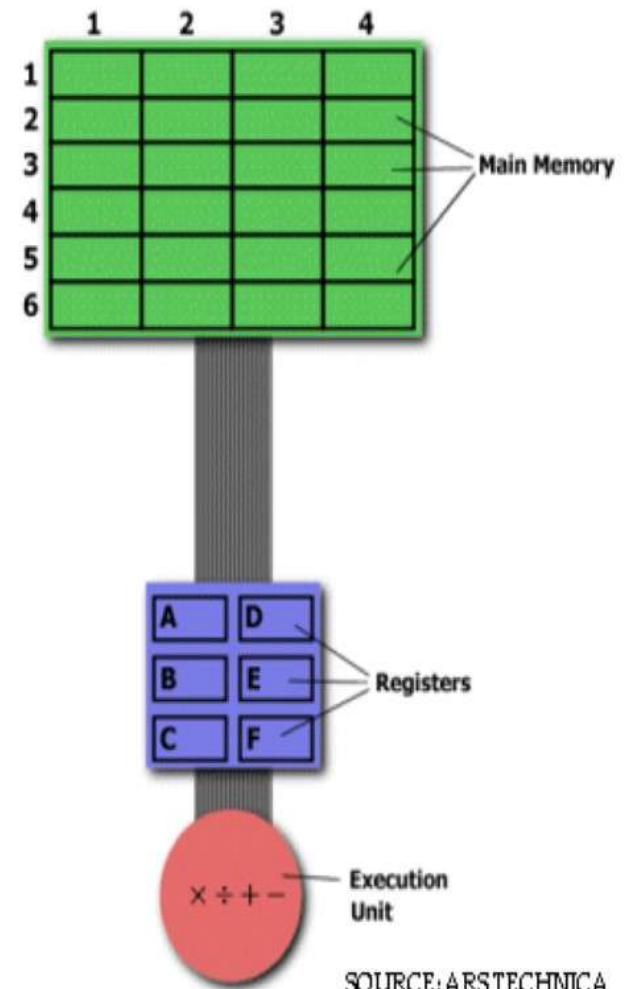
- Multiplying Two Numbers in Memory**

On the right is a diagram representing the storage scheme for a generic computer.

The main memory is divided into locations numbered from (row) 1: (column) 1 to (row) 6: (column) 4.

The execution unit is responsible for carrying out all computations. However, the execution unit can only operate on data that has been loaded into one of the six registers (A, B, C, D, E, or F).

Let's say we want to find the product of two numbers - one stored in location 2:3 and another stored in location 5:2 - and then store the product back in the location 2:3.



SOURCE: ARSTECHNICA



The CISC Approach

The primary goal of CISC architecture is to **complete a task in as few lines** of assembly as possible. This is achieved by building processor hardware that is capable of understanding and executing a series of operations. For this particular task, a CISC processor would come prepared with a specific instruction (we'll call it "MULT"). When executed, this instruction loads the two values into separate registers, multiplies the operands in the execution unit, and then stores the product in the appropriate register. Thus, the entire task of multiplying two numbers can be completed with one instruction:

MULT 2:3, 5:2

MULT is what is known as a "complex instruction." It operates **directly on the computer's memory banks** and **does not require** the programmer to explicitly call any loading or storing functions. It closely resembles a command in a higher level language. For instance, if we let "a" represent the value of 2:3 and "b" represent the value of 5:2, then this command is identical to the C statement "a = a * b."

One of the **primary advantages** of this system is that the compiler has to **do very little work** to translate a high-level language statement into assembly. Because the length of the code is relatively short, **very little RAM is required** to store instructions. The emphasis is put on **building complex instructions directly into the hardware**.



The RISC Approach

RISC processors only use **simple instructions** that can be executed within **one clock cycle**. Thus, the "MULT" command described above could be divided into three separate commands: "**LOAD**," which moves data from the memory bank to a register, "**PROD**," which finds the product of two operands located within the registers, and "**STORE**," which moves data from a register to the memory banks. In order to perform the exact series of steps described in the CISC approach, a programmer would need to code four lines of assembly:

```
LOAD A, 2:3  
LOAD B, 5:2  
PROD A, B  
STORE 2:3, A
```

At first, this may seem like a much less efficient way of completing the operation. Because there are more lines of code, more RAM is needed to store the assembly level instructions. The compiler must also perform more work to convert a high-level language statement into code of this form.

However, the RISC strategy also brings some very important **advantages**. Because each instruction requires **only one clock cycle** to execute, the entire program will execute in approximately the same amount of time as the multi-cycle "MULT" command. These RISC "reduced instructions" **require less transistors of hardware space** than the complex instructions, **leaving more room** for general purpose registers. Because all of the instructions execute in a uniform amount of time (i.e. one clock), **pipelining is possible**.

Separating the "LOAD" and "STORE" instructions actually **reduces the amount of work that the computer must perform**. After a CISC-style "MULT" command is executed, the processor automatically erases the registers. If one of the operands needs to be used for another computation, the processor must re-load the data from the memory bank into a register. In RISC, the operand will remain in the register until another value is loaded in its place.



Conclusion

CISC

- Emphasis on hardware
- Includes multi-clock complex instructions
- Memory-to-memory: "LOAD" and "STORE" incorporated in instructions
- Small code sizes, high cycles per second
- Transistors used for storing complex instructions

RISC

- Emphasis on software
- Single-clock, reduced instruction only
- Register to register: "LOAD" and "STORE" are independent instructions
- Low cycles per second, large code sizes
- Spends more transistors on memory registers